

Music Streaming App MVP

Status:

Tags: android, spring-boot, music, streaming

What MVP Means Here

A user can:

- Open the app and play music without signing in
 - Choose biometric or password auth on first launch
 - Browse, search, filter music by genre
 - Stream music from backend OR play downloaded/local files
 - Create / edit / rate playlists (authenticated)
 - Upload music files, server transcodes for streaming (authenticated)
 - Keep playback running in background with media-style notification
 - See synced subtitles (server-generated via AI if missing)
 - Add community providers via manifest repos
 - Copy another user's playlist and make it their own
-

Tech Stack

Mobile (Android)

- Room DB + KSP
- Jetpack Compose + Navigation
- Kotlin Coroutines + Flow + ViewModel
- Hilt (DI)
- AndroidX Security (EncryptedSharedPreferences, MasterKey)
- AndroidX Biometric
- Retrofit + OkHttp + Kotlinx Serialization

- Media3 (ExoPlayer + MediaSession + MediaSessionService) — playback + notification + background
- WorkManager — download jobs, sync jobs
- Coil — artwork
- DataStore — settings (non-sensitive)
- Accompanist Permissions

Backend (Spring Boot)

- Spring Boot 3 + Kotlin
- Spring Security + JWT (stateless)
- Spring Data JPA
- **PostgreSQL** — metadata (users, tracks, playlists, ratings, providers). Relational, full-text search, mature
- **S3 / MinIO** — audio blobs (original + transcoded variants)
- **Redis** — rate limiting, short-lived caches, refresh tokens, pub/sub
- **RabbitMQ** — job queue for transcoding + subtitle generation (long-running, retrievable, back-pressure friendly)
- **FFmpeg** — transcoder worker (HLS + 128/256 kbps Opus/AAC)
- **Whisper (faster-whisper)** — AI subtitle/lyric alignment worker
- **Flyway** — migrations
- **Bucket4j** (Redis-backed) — rate limiting filter

Why queue + rate limiting: upload/transcode is CPU-heavy and unbounded in duration — must be async via queue so HTTP doesn't time out and workers scale horizontally. Rate limit on upload, stream-start, and auth endpoints to stop abuse and protect transcode workers.

Module Layout (Mobile)

```
:app
:core:designsystem
:core:common
:core:database      (Room, local cache)
:core:datastore     (settings)
```

```
:core:network      (Retrofit, interceptors)
:core:security     (crypto, biometric gate)
:core:player       (Media3 service + controller)
:core:providers    (provider SPI + manifest loader)
:feature:auth
:feature:home
:feature:search
:feature:player
:feature:playlist
:feature:library
:feature:upload
:feature:providers
:feature:settings
```

MVP Features

Feature 1: Project Setup + Room DB Foundation

Everything else builds on this.

Steps:

1. Create Android project — Empty Activity, Kotlin, API 26, Compose
2. Add dependencies listed above; enable KSP; Hilt plugin
3. Create module layout above
4. Create Room entities:
 - `TrackEntity` — id, remotId, title, artist, album, genre (enum), durationMs, artworkUrl, localPath (nullable), streamUrl (nullable), isDownloaded, providerId, updatedAt
 - `PlaylistEntity` — id, remotId, name, ownerId, isMine, isCopy, sourcePlaylistId (nullable), ratingAvg, updatedAt
 - `PlaylistTrackCrossRef` — playlistId, trackId, position
 - `SubtitleLineEntity` — id, trackId, startMs, endMs, text
 - `DownloadEntity` — id, trackId, state, progress, filePath
 - `PendingOpEntity` — id, type, payloadJson, createdAt (outbox for sync)
 - `ProviderEntity` — id, manifestUrl, name, version, capabilities, enabled

- `UserEntity` — id, username, email, tokenEnc, refreshEnc (nullable if anon)
 - `RatingEntity` — id, targetType (TRACK/PLAYLIST), targetId, value, syncedAt
5. Create DAOs (Flow-returning reads): `TrackDao`, `PlaylistDao`, `SubtitleDao`, `DownloadDao`, `PendingOpDao`, `ProviderDao`, `UserDao`, `RatingDao`
 6. Create `AppDatabase` with type converters (Genre enum, Instant)
 7. Create `Genre` enum — POP, ROCK, HIPHOP, RNB, ELECTRONIC, CLASSICAL, JAZZ, METAL, FOLK, COUNTRY, LATIN, OTHER (fixed set)
 8. Test — insert fake track/playlist, read back

Done when: DB reads/writes verified via unit test.

Feature 2: Startup Auth Gate (Biometric or Password)

First launch asks user: biometric or regular auth. Anonymous use allowed (no sign-in).

Steps:

1. `AuthPreference` in DataStore — `authMode` ∈ { NONE, PASSWORD, BIOMETRIC }, `skippedAuth` bool
2. Build `WelcomeScreen` shown on first launch only:
 - "Sign in with biometric"
 - "Sign in with password"
 - "Skip — browse anonymously"
3. Build `AuthViewModel`:
 - On biometric: check `BiometricManager.canAuthenticate()`. If not enrolled → disable button with hint
 - On password: normal email/password flow → backend `/auth/login` → JWT (access + refresh)
4. `SecureTokenStore` (AndroidX Security) — store refresh token + derived key in `EncryptedSharedPreferences` backed by `MasterKey` (StrongBox when available)

5. Biometric unlock flow:

- `BiometricPrompt` gates decryption of refresh token via `BiometricPrompt.CryptoObject` (AES key in Keystore with `setUserAuthenticationRequired(true)`)
- On success → decrypt refresh → silent refresh access token
- Skip the biometric prompt entirely on later launches **only** if feature toggled on AND user previously opted in

6. Guard: protected actions (create/edit/delete playlist, upload, rate) check

`authState.isAuthenticated`; if false → prompt login

Done when: First launch picks mode → subsequent launches auto-unlock (or skip) per choice → protected actions gated.

Feature 3: Player Core (Media3 + Background + Notification)

Playback continues when app is closed if media is playing.

Steps:

1. Create `MusicService : MediaSessionService`:
 - Holds `ExoPlayer` + `MediaSession`
 - Foreground service with `mediaPlayback` type (required API 34+)
 - Declares `FOREGROUND_SERVICE_MEDIA_PLAYBACK` + `POST_NOTIFICATIONS` perms
 - Provides `MediaLibrarySession` for queue, next/prev, seek, shuffle, repeat
2. `PlayerRepository` — exposes `MediaController` flow to UI
3. Notification auto-handled by Media3 with media controls (play/pause/next/prev/seek)
4. Source resolver: if `track.localPath` present or `isDownloaded` → local file; else stream URL from provider/backend (HLS via `HlsMediaSource.Factory`)
5. Handle audio focus, become-noisy, headset events (Media3 default)
6. Background rule: service promotes to foreground while `isPlaying`; demotes on pause > N minutes or stop → service can be killed by system safely
7. Build `NowPlayingScreen` + mini player — binds to `MediaController`

Done when: App swiped away while playing → music continues → notification shows → controls work → reopening app restores state.

Feature 4: Browse, Search, Filter by Genre

Steps:

1. `HomeScreen` — featured playlists, recently played, trending (served from backend, cached to Room)
2. `SearchScreen` :
 - Query field (debounced 300 ms)
 - Genre filter chip row from `Genre` enum (fixed)
 - Results: tracks + playlists tabs
 - Backend endpoint `GET /search?q=&genre=` (Postgres `tsvector`)
3. Offline fallback — query Room when no network
4. `SearchViewModel` uses `stateIn` + `SavedStateHandle` to survive config changes

Done when: Typing query returns results, genre chips filter correctly, offline shows cached results.

Feature 5: Playlists (Create / Edit / Copy / Rate)

Steps:

1. `LibraryScreen` lists user playlists + saved/copied ones
2. `PlaylistDetailScreen` — track list with drag-to-reorder, add/remove, play-all, shuffle toggle
3. **Intelligent shuffle** (no AI):
 - Fisher–Yates with constraints:
 - No two consecutive tracks by same artist
 - Genre clustering reduced (weighted spread)
 - Previously played in last K items demoted
 - Implemented in `:core:player` as `ShuffleOrder` custom impl

4. **Copy playlist:** `POST /playlists/{id}/copy` → server clones metadata + tracks
→ returns new playlist owned by caller; mobile mirrors to Room with
`isCopy=true` , `sourcePlaylistId=original`
5. **Rating:** 1–5 stars on tracks and playlists, `RatingEntity` local + `POST /ratings`
remote; aggregate avg shown on detail
6. Permission check: all mutations require auth

Done when: Create playlist → add tracks → reorder → copy someone else's → edit copy → rate → see updated average.

Feature 6: Downloads + Offline Playback

Steps:

1. `DownloadScreen` / download button on track row
2. `DownloadWorker` (WorkManager, `expedited` if API allows):
 - Fetch stream URL → download to app-private storage
 - Update `DownloadEntity` progress → Flow drives UI
 - On finish → set `TrackEntity.isDownloaded=true` , `localPath=...`
 - Post notification "Download complete: {title}" (separate notification channel from playback)
3. Player source resolver already prefers local when present (Feature 3)
4. Delete download flow — remove file + flags

Done when: Download track → airplane mode → play it from Library.

Feature 7: Upload + Server Transcoding

Users with auth upload music. Server transcodes to streaming-optimized variants.

Mobile steps:

1. `UploadScreen` — file picker (SAF), metadata fields (title, artist, album, genre enum), artwork picker
2. `POST /uploads` — multipart upload (show progress via OkHttp listener)
3. Response: `uploadId` , `status=PROCESSING`

4. Poll or SSE `GET /uploads/{id}` until `READY`; then track appears in library

Backend steps:

1. Endpoint stores blob to S3/MinIO `raw/{uploadId}`, writes `Track{status=PROCESSING}`, enqueues RabbitMQ `transcode.requested`
2. **Transcode worker** (separate JVM service):
 - Consumes message, pulls raw blob
 - FFmpeg → HLS segments (Opus 96k + AAC 128k + AAC 256k) → write to `stream/{trackId}/`
 - Extract duration, replay-gain, waveform peaks
 - On success emit `transcode.completed` → API updates `Track.status=READY`, sets `streamUrl` (HLS master)
 - On failure emit to DLQ after N retries
3. **Subtitle worker**:
 - On `transcode.completed` enqueue `subtitles.requested` if no lyrics provided
 - Worker runs faster-whisper → writes `SubtitleLineEntity` rows → emit `subtitles.ready`
4. Rate limit `POST /uploads` — e.g. 10/hour/user via Bucket4j+Redis

Done when: Upload MP3 → server returns READY → track plays via HLS → subtitles appear later.

Feature 8: Subtitles (Follow the Music)

Steps:

1. `GET /tracks/{id}/subtitles` → list of lines
2. Cache to Room on first play
3. `SubtitleOverlay` Composable subscribes to `player.currentPositionMs` (200 ms poll via `flow`) → highlights active line, auto-scrolls
4. If response empty → show "Generating..." banner; poll/SSE until ready

Done when: Playing a track with subtitles scrolls in sync; uploaded track without lyrics eventually gets AI-generated ones.

Feature 9: Sync Strategy (Local ↔ Remote)

Two DBs → potential conflict. Resolve via **outbox + last-write-wins with version vectors for playlists**.

Rules:

1. All mutations write to Room first, then insert a `PendingOpEntity` (type + payload). UI updates instantly
2. `SyncWorker` (periodic + on-connectivity) drains outbox, calls backend, marks ops applied
3. Each server entity carries `updatedAt` + `version` (monotonic int). Pull sync uses `GET /changes?since=cursor`
4. Conflict rules:
 - **Ratings** — last-write-wins (trivial)
 - **Playlist metadata (name, order)** — server `version` is authoritative; if local op based on stale version, server rejects → client re-fetches, re-applies op on top, retries. Resolver surfaces rare merge conflicts to user
 - **Track metadata** — server authoritative (users don't edit track meta post-upload in MVP)
 - **Uploads / deletes** — idempotent via client-generated UUIDs
5. Anonymous users keep everything local; on login, outbox flushes under new userId

Done when: Edit playlist offline → reconnect → change appears on server; concurrent edits resolved without data loss.

Feature 10: Community Providers

Modular plugin system — a provider is described by a manifest hosted in a git repo (or any HTTPS URL).

Manifest (`provider.json`):

```
{
  "id": "com.example.spotify-bridge",
  "name": "Spotify Bridge",
```

```

    "version": "1.0.0",
    "author": "...",
    "capabilities": ["search", "stream", "playlists"],
    "authType": "oauth2",
    "authConfig": { "authUrl": "...", "tokenUrl": "...", "scopes": ["..."] },
    "baseUrl": "<https://provider.example.com/api>",
    "endpoints": {
        "search": "/search?q={q}&genre={genre}",
        "track": "/tracks/{id}",
        "stream": "/tracks/{id}/stream"
    },
    "signature": "..."
}

```

Mobile steps:

1. `:core:providers` defines `MusicProvider` SPI: `search()`, `getTrack()`, `getStreamUrl()`, `listPlaylists()` — all suspend, return common DTOs
2. `ProviderRegistry` keeps instances keyed by `ProviderEntity.id`; `BackendProvider` is built-in, always enabled
3. `ProviderManifestLoader` — user pastes manifest URL → app downloads JSON → validates schema + signature → stores in Room → instantiates `ManifestProvider` (data-driven Retrofit client reading endpoints from manifest)
4. `ProvidersScreen` — list / add / toggle / remove
5. OAuth flow via Chrome Custom Tabs; tokens stored in `EncryptedSharedPreferences` per-provider
6. Search and browse aggregate across enabled providers; track origin tagged by `providerId`
7. Self-hosted providers = same mechanism, any HTTPS endpoint that honors the SPI

Security:

- Manifest signature check against an allow-list of public keys OR user-accept-on-add (shows capabilities first)

- Providers run **data-only** (no code exec) in MVP — all calls go through a hardened Retrofit builder with strict timeouts and response-size caps

Done when: Add manifest URL → provider appears → search returns its results → play a track from it.

Backend MVP Endpoints (summary)

```

POST    /auth/register
POST    /auth/login                (rate-limited)
POST    /auth/refresh

GET     /tracks/{id}
GET     /tracks/{id}/subtitles
GET     /search?q=&genre=

POST    /uploads                  (multipart, rate-limited)
GET     /uploads/{id}

GET     /playlists/{id}
POST    /playlists
PATCH  /playlists/{id}
POST    /playlists/{id}/copy
POST    /playlists/{id}/tracks

POST    /ratings                  (target=TRACK|PLAYLIST)

GET     /changes?since=cursor      (sync pull)

```

Every write endpoint: JWT required, Bucket4j/Redis rate limit, idempotency key supported.

What This MVP Demonstrates

Concept	Where
Room + KSP	Track, Playlist, Subtitle, PendingOp, Rating
Reactive UI	Flow → Compose for library, downloads, player
Hilt DI	All repos, services, use cases
AndroidX Security	Token + provider-credential storage
Biometric	Gated refresh-token decryption
Media3 background playback	Foreground service + notification

Concept	Where
WorkManager	Downloads, sync
Retrofit	Backend + manifest-driven providers
Offline-first + sync	Outbox + versioned pull
Queue + rate limiting	Upload → transcode → subtitles
AI augmentation	Whisper subtitle generation
Modularity	Gradle modules + provider SPI

Post-MVP Additions

Addition	Value
Social (follow, feed)	Network effects
Collaborative playlists	Real-time via WebSocket
Lyrics karaoke mode	Engagement
Recommendations	Retention
Cast to Chromecast	Polish
Equalizer	Polish
Android Auto	Reach
CDN for HLS	Scale
Signed provider marketplace	Trust
E2E-encrypted self-hosted lib	Privacy